INTERNSHIP REPORT

ON

FULL-STACK DEVELOPMENT

Submitted To

School of Computer Science and Engineering IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of
B.Tech CSE

By

AKSHAT MAURYA

Roll Number: 2410030668 Batch:
2024–2028

2026–27

CANDIDATE'S DECLARATION

I, Akshat Maurya, hereby declare that the internship report titled "Full-Stack Development Internship and TaskFlow Web Application" has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering. The report describes the learning, practical activities and project work undertaken during my internship period. References used for technical concepts have been acknowledged in the bibliography. I further declare that this report has not been submitted to any other institute for any degree or diploma requirement and that I shall be responsible for any copyright violation arising from material reproduced without proper acknowledgement.

Signature: _____

Student Name: Akshat Maurya

Roll No.: 2410030668

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to everyone who supported me during my Full-Stack Development Internship. The internship provided me with practical exposure to frontend development, backend APIs, database integration, debugging, testing and the overall software development lifecycle.

I am thankful to UniConverge Technologies and upSkillCampus for providing the internship opportunity and hands-on learning environment. I also acknowledge the guidance associated with The IoT Academy and the mentors who helped me understand how individual programming concepts are combined to develop a complete web application.

I am grateful to the School of Computer Science and Engineering, IILM University, Greater Noida, for providing the academic framework in which this internship was completed. Finally, I express my gratitude to my parents and everyone who encouraged me throughout the internship.

Signature: _____

Student Name: Akshat Maurya

Roll No.: 2410030668

INTERNSHIP COMPLETION CERTIFICATE

Certificate issued by the internship organization

Internship period: 02-08-2026 to 30-08-2026

Domain: Full-Stack Development

TABLE OF CONTENTS

4. PROJECT DESCRIPTION

4.1 Introduction

The internship was completed in the domain of Full-Stack Development from 02 August 2026 to 30 August 2026. The internship focused on understanding how a modern web application is designed as an integrated system rather than as separate frontend and backend programs.

As a practical project, a web-based task and project management application named "TaskFlow" was designed. The application allows users to create, view, update and delete tasks, assign priorities and status values, and monitor task progress through a dashboard. The project was selected because it covers the major responsibilities of a fullstack developer: user interface development, client-server communication, REST API design, database operations, validation and testing.

The project follows a three-layer structure: a React-based presentation layer, an Express/Node.js application layer and a MongoDB data layer. The separation of responsibilities makes the application easier to understand, maintain and extend.

Project Summary

Item	Details
Project Title	TaskFlow – Full-Stack Task & Project Management Web Application
Internship Domain	Full-Stack Development
Student	Akshat Maurya
Roll Number	2410030668
Internship Period	02-08-2026 to 30-08-2026
Application Type	Responsive web application
Primary Objective	Practice complete frontend-to-database application development
Core Modules	Dashboard, Task CRUD, Task status, Priority, REST API, Database

4.2 Organization Profile

The internship certificate identifies UniConverge Technologies and upSkillCampus in connection with the internship program and records hands-on training and real industry project work in the domain of Full-Stack Development. The certificate also carries references to the All India Council for Technical Education (AICTE) and The IoT Academy. The internship was completed during the period 02-08-2026 to 30-08-2026.

The training environment was oriented toward practical software development. The emphasis of the internship was on applying programming knowledge to a complete application workflow: planning requirements, designing the user interface, implementing APIs, connecting the application with a database, handling errors and validating the final application.

Learning Environment

- Frontend development using component-based design.
- Backend development using Node.js and Express.
- REST API development and HTTP request/response handling.
- Database modelling and CRUD operations using MongoDB.
- Use of Git and GitHub for source-code version control.
- Debugging, validation and testing of application features.
- Understanding how frontend, backend and database layers communicate.

Internship Certificate Details

Field	Recorded Detail
Candidate	Akshat Maurya
Domain	Full-Stack Development
Program	Internship Program
Start Date	02-08-2026
End Date	30-08-2026
Certificate ID	USC704188TIA
Certificate Signatory	Kaushlendra Singh Sisodia

4.3 Problem Statement

Small student and team projects often use scattered notes, spreadsheets or chat messages to track work. This makes it difficult to maintain a single view of pending, active and completed tasks. A lightweight web application is therefore required to centralize task information and provide simple operations for managing work.

The technical problem is to build an end-to-end application in which a browser-based interface can communicate reliably with a backend service, while the backend validates requests and stores persistent data in a database. The system should also provide meaningful feedback when invalid data or unavailable resources are encountered.

4.4 Project Objectives

- Design a responsive and easy-to-use task management interface.
- Develop RESTful APIs for task creation, retrieval, modification and deletion.
- Connect the backend to MongoDB for persistent storage.
- Implement validation for required task fields and supported status/priority values.
- Display task statistics and progress information on a dashboard.
- Practice separation of frontend, backend and database responsibilities.
- Use Git-based version control and document the project for future extension.

4.5 Scope of the Project

The scope includes the complete prototype from browser interface to database. A user can create a task with a title, description, priority and status; view all tasks; update task information; delete a task; and filter tasks by status or priority. The dashboard summarizes the number of tasks in different states.

The project is intentionally scoped as an internship-level prototype. Advanced production features such as enterprise role management, real-time collaboration, payment processing and large-scale cloud deployment are outside the current scope but can be added later.

Functional Requirements

ID	Requirement
FR-01	Create a new task.
FR-02	Display all saved tasks.
FR-03	Update task status, priority and other fields.

FR-04	Delete a task.
FR-05	Filter tasks by status or priority.
FR-06	Show task statistics on dashboard.
FR-07	Return clear error messages for invalid requests.

4.6 Technologies and Tools Used

Technology / Tool	Purpose
HTML5	Page structure and semantic content.
CSS3	Layout, responsiveness and visual styling.
JavaScript	Application logic and browser-side interaction.
React.js	Component-based frontend development.
Node.js	JavaScript runtime for backend development.
Express.js	REST API and HTTP server framework.
MongoDB	Document-oriented database for persistent task data.
Mongoose	MongoDB object modelling and validation.
Git	Version control and source-code tracking.
GitHub	Remote repository and collaboration workflow.
VS Code	Development environment.
Postman	API testing and request validation.

Development Environment

The project was structured as separate frontend and backend directories. The frontend is responsible for rendering components and sending HTTP requests. The backend exposes API endpoints and performs validation and database operations. Environment variables are used for configuration such as the MongoDB connection string and server port.

Major API Endpoints

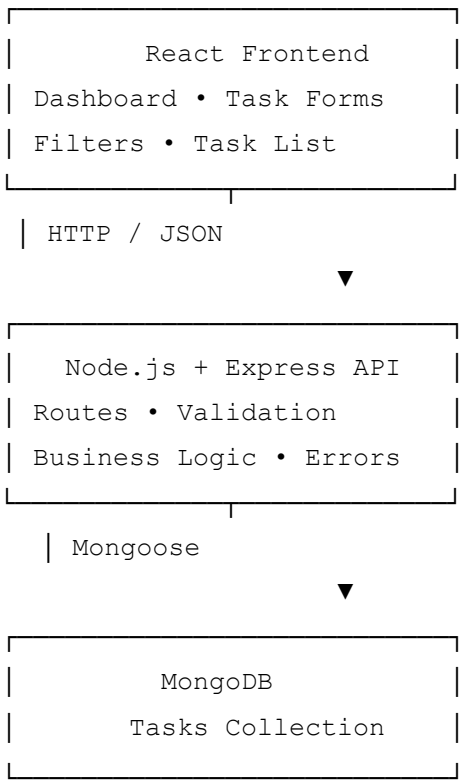
Method	Endpoint	Purpose
GET	/api/tasks	Fetch all tasks
POST	/api/tasks	Create a task
PUT	/api/tasks/:id	Update a task
DELETE	/api/tasks/:id	Delete a task
GET	/api/health	Check server status

Data Model

Field	Type	Description
title	String	Short task name; required
description	String	Detailed task information
priority	String	Low, Medium or High
status	String	Todo, In Progress or Done
createdAt	Date	Creation timestamp
updatedAt	Date	Last modification timestamp

4.7 System Architecture

The system uses a client-server architecture. The React frontend runs in the browser and communicates with the Express API over HTTP. The Express server contains routing, validation and business logic. Mongoose is used by the server to communicate with MongoDB.



4.7.1 Frontend Layer

The frontend provides the user-facing interface. Components are kept focused on individual responsibilities such as task forms, task cards, filters and dashboard statistics.
React state is used to refresh the interface after API operations.

4.7.2 Backend Layer

The backend provides REST endpoints. Incoming data is checked before it is stored. Successful operations return JSON responses, while invalid input or missing records produce appropriate HTTP status codes and error messages.

4.7.3 Database Layer

MongoDB stores each task as a document. The schema defines the expected fields and allowed values. Timestamps make it possible to determine when records were created or updated.

4.7.4 Request Flow

- User performs an action in the React interface.
- React sends an HTTP request to the Express API.
- Express validates the request and identifies the required operation.
- Mongoose reads or modifies the corresponding MongoDB document.
- The backend returns a JSON response.
- React updates the visible task list and dashboard.

4.8 Methodology

The project was developed using an incremental methodology. Instead of implementing every feature at once, the application was divided into small modules and each module was tested before moving to the next stage.

Phase 1 – Requirement Analysis

The initial requirement was translated into a small set of practical features: task creation, task listing, editing, deletion, filtering and dashboard statistics. This reduced the project to a manageable full-stack workflow while still covering the complete development stack.

Phase 2 – UI Design

A simple dashboard layout was planned with a navigation area, summary cards, task form and task list. The design emphasizes readable task information and quick status changes. Responsive CSS was included so the interface can adapt to smaller screens.

Phase 3 – Backend Development

The Express server was configured with JSON parsing and API routes. Controllers perform CRUD operations and return structured JSON. Basic validation prevents empty titles and unsupported status or priority values.

Phase 4 – Database Integration

A Mongoose model was created for tasks. The API was connected to MongoDB through an environment variable. Database errors are caught and converted into understandable API responses.

Phase 5 – Frontend Integration

The React application uses the Fetch API to communicate with the backend. After create, update or delete operations, the task list is refreshed so that the displayed state remains consistent with the database.

Phase 6 – Testing and Debugging

API endpoints were tested with valid and invalid requests. Frontend interactions were checked for successful loading, empty states and error responses. Common issues

considered during debugging included incorrect API paths, malformed JSON, missing database configuration and invalid object identifiers.

Testing Table

Test Case	Input / Action	Expected Result
TC-01	Open dashboard	Dashboard loads and requests task data.
TC-02	Create task with valid title	Task is stored and shown in list.
TC-03	Create task with empty title	Validation error is displayed.
TC-04	Edit task status	Updated status appears in list.
TC-05	Delete existing task	Task disappears after successful deletion.
TC-06	Filter by priority	Only matching tasks are displayed.
TC-07	Backend health request	API returns successful health response.

4.8.1 Key Features Implemented

- Task CRUD operations through REST APIs.
- Task status workflow: Todo → In Progress → Done.
- Priority levels: Low, Medium and High.
- Dashboard summary of task counts.
- Search/filter controls for task organization.
- Responsive frontend layout.
- Backend validation and error handling.
- Persistent storage using MongoDB.
- Modular code organization for future maintenance.

4.8.2 Security and Reliability Considerations

Although the project is an internship prototype, basic secure-development practices were considered. Database credentials are not hard-coded into source files and are intended to be supplied through environment variables. API input is validated before database operations. In a production deployment, authentication, authorization, HTTPS, rate limiting, stronger validation and centralized logging should be added.

4.8.3 Challenges Encountered and Learning

Challenge	Learning / Resolution
Connecting frontend and backend	Understood API URLs, HTTP methods and JSON responses.
Keeping UI data synchronized	Refreshed or updated React state after API operations.
Database validation	Used schema rules and server-side checks.
Handling errors	Returned useful status codes and messages.
Organizing code	Separated components, routes, models and configuration.
Debugging requests	Used browser developer tools and API testing tools.

4.8.4 Suggested Future Enhancements

- JWT-based login and role-based access control.
- User-specific task ownership and assignment.
- Due dates, reminders and calendar integration.

- Real-time updates using WebSockets.
- File attachments and activity history.
- Automated unit and integration testing.
- Cloud deployment with CI/CD.
- Analytics for project completion and productivity.

4.9 Expected Outcomes

The expected outcome of the internship project is a working full-stack prototype that demonstrates the complete flow from a browser interface to persistent database storage. The project is designed to show practical understanding rather than only theoretical knowledge.

Technical Outcomes

- Ability to build reusable React components.
- Ability to create and consume REST APIs.
- Understanding of Node.js and Express server structure.
- Practical experience with MongoDB data storage.
- Understanding of CRUD operations and validation.
- Experience with debugging and API testing.
- Basic understanding of version-controlled development.

Internship Learning Outcomes

The internship strengthened the understanding that full-stack development requires coordination between multiple layers. A frontend feature is only complete when its interface, API contract, validation and database behaviour are consistent. The project also provided experience in breaking a larger problem into smaller modules, testing each module and documenting the final workflow.

Conclusion

The Full-Stack Development Internship provided practical exposure to modern web application development. Through the TaskFlow project, the major concepts of frontend development, backend API design and database integration were combined into one application. The completed prototype provides a foundation for adding authentication, collaboration, analytics and deployment features in future work.

4.10 Internship Evidence and Communication Proof

The internship completion certificate supplied with this report is attached in Chapter 3. It records the candidate as Akshat Maurya, identifies the Full-Stack Development domain, specifies the internship period as 02-08-2026 to 30-08-2026 and provides Certificate ID

USC704188TIA. The certificate is the primary completion evidence available for inclusion in this report.

upSkill CAMPUS

C-56/11, Ground floor, Near IT Stellar Park, Sector-62, Noida, U.P. - 201309
Email: support@upskillcampus.com, Ph. No. 9354068856

Internship Offer Letter

Date: **23-08-2026**

Name: **Akshat Maurya**
Email: **akshat.maurya.cs28@iilm.edu**
Location: **Remote**

Dear **Akshat Maurya** ,

We are pleased to inform you that **Upskill Campus** alongwith its industry Partner **UniConverge Technologies Pvt. Ltd.** has agreed to provide you internship as **Full-Stack Developer Intern** Your appointment of service is based on the following terms and conditions.

1. The internship shall start from **02-08-2026**
2. **Internship:** The internship period will be 4 to 6 weeks, starting from the date of joining.
3. **Working Hours:** The internship runs from 10:00 AM to 6:00 PM.

On behalf of both the Company, we wish you every success in your position and trust that our relationship will be long and mutually rewarding.

Sincerely
For Upskill Campus and UniConverge Technologies Pvt. Ltd.

Prabha Singh
HR Manager

5. BIBLIOGRAPHY / REFERENCES

[1] Mozilla Developer Network (MDN), JavaScript and Web Development documentation.

[2] React Documentation, React component and state management concepts.

[3] Node.js Documentation, Node.js runtime and server-side JavaScript concepts.

[4] Express.js Documentation, routing and middleware concepts for web servers.

[5] MongoDB Documentation, document database and CRUD concepts.

[6] Mongoose Documentation, MongoDB object modelling and schema validation.

[7] Git Documentation, version control and repository management concepts.